

Hosting facts (PDPL boundary, regions, domains)

SECTION 06-operations-it

TYPE document

STATUS **active**

OWNER Founder

UPDATED 2026-08-20

Single-page reference for where Fly GACA actually runs. Everything else in `06-operations-it/` keys off this page — check it before reaching for the longer planning docs, and before restating a region, a datastore or a repo path anywhere else in the tree.

[!CAUTION] **Target, not deployed — read this before quoting any region from this page.**

The Express-on-Cloud-Run / Cloud SQL / `me-central2` stack described below is the **intended** architecture. It has **never been deployed**. Verified against `gcloud` on **2026-08-19**: no `flygaca-api` Cloud Run service exists in any project. Production is still the **previous Firebase Functions stack**, running as 14 individual Cloud Run services in project `flygaca-sa`, **all in `me-central1` (Doha, Qatar)**, with Cloud SQL in `us-east4` (**Northern Virginia**). `me-central2` has **not been granted** to this Google account — `gcloud` returns *"Permission denied on 'locations/me-central2' ... Access to the region is unavailable. Please contact our sales team."* The migration is blocked on that region grant.

Customer data is therefore NOT in-Kingdom today. Every section below is marked **[TARGET]** or **[AS BUILT]** — do not restate a `me-central2` value as current fact.

[AS BUILT] What is actually deployed today (verified 2026-08-19)

Concern	What runs it	Region
Project	<code>flygaca-sa</code>	—
API + app	14 individual Cloud Run services, the previous Firebase Functions build — <code>createcheckoutconfig</code> , <code>confirmpayment</code> , <code>moyasarwebhook</code> , <code>chat</code> , <code>provisionseats</code> , <code>renewmoyasarsubscriptions</code> , <code>claimschoolseat</code> , <code>claimstaffaccess</code> , <code>claimfoundingaccess</code> , <code>getcohortreadiness</code> , <code>getmyorgs</code> , <code>getreferralcode</code> , <code>cancelautorenew</code> , <code>flygaca-app</code>	<code>me-central1</code> — Doha, Qatar (NOT in-Kingdom)
Datastore	Cloud SQL <code>flygaca-sa-instance</code> (PostgreSQL 18)	<code>us-east4</code> — Northern Virginia, USA (NOT in-Kingdom)
Second Cloud SQL instance	<code>flygaca-fdc</code>	<code>me-west1</code> — Tel Aviv (NOT in-Kingdom)

Concern	What runs it	Region
Express service flygaca-api	Does not exist in any project — never deployed	—
me-central2 (Dammam)	Not available to this Google account — permission denied, region grant pending with Google sales	—
Prices	Old card, old env names — MOYASAR_PRICE_*_SAR on the Functions services (see secrets-and-keys-placement.md)	—

Blocker to clear before any of the [TARGET] sections below become true: a Google grant for me-central2 , then the Express deploy and the Cloud SQL migration.

[TARGET] The stack in one table

Concern	What runs it
SPA (dist/)	Cloud Storage bucket behind an HTTPS load balancer
API (server/)	Cloud Run service — a single Express 5 app, region me-central2
Datastore	Cloud SQL for PostgreSQL, same region
Sessions	HS256 JWT in an HttpOnly cookie, signed by the API (SESSION_SECRET)
Sign-in	Email + password (scrypt) and Google, via server-side OAuth
Payments	Moyasar — mada, Apple Pay, cards (hosted widget + server-to-server confirm)
Renewals	Cloud Scheduler → POST /api/billing/renew , carrying CRON_SECRET
AI (Captain Adel)	Gemini via Genkit, RAG over the GACAR corpus
Transactional email	Any Resend-compatible endpoint (MAIL_ENDPOINT / MAIL_API_KEY)
Secrets	Google Secret Manager, mounted into the Cloud Run revision

[!IMPORTANT] There is no Firebase in the current ay2m/FLyGACA codebase — no Hosting, no Auth, no Functions, no Firestore, no App Check. Auth, the datastore, the API and hosting are all first-party or plain GCP *in the source tree*. That is a statement about the code, not about production: what is deployed today is still the previous Firebase Functions build (see the [AS BUILT] table above). Payments are Moyasar, never Stripe — that one is true in both the code and production.

Domains

Domain	Status	Notes
<code>flygaca.com</code>	Owned, canonical	The main product. Marketing domains consolidate here.
<code>api.flygaca.com</code>	The API origin	[TARGET] — the single Express Cloud Run service, once deployed. Mirrors proxy <code>/api/*</code> here. Today the app still calls the Functions-era services in <code>flygaca-sa</code> .
<code>captadel.com</code>	Owned (secured 2026-05-23)	Captain Adel's marketing / standalone front door; redirects to <code>flygaca.com</code> for traffic still hitting the Vercel mirror. Registrar not yet recorded — write it in here when known.
<code>flygaca.sa</code>	Unblocked (entity exists) — see <code>flygaca-sa-registration-readiness.md</code>	Register after the P0-2 name opinion; the "gaca" substring may trigger SaudiNIC name review. Redirect-only → <code>flygaca.com</code> .

PDPL data boundary — the load-bearing rule (policy) vs where data sits (fact)

The **policy** is unchanged:

All personal data — accounts, profiles, the logbook, **real user queries** — stays in the Kingdom (Cloud SQL + Cloud Run, `me-central2`). Public corpus work can happen on the EU VPS. — `phase0.md`, P0-6, restated for the current stack

[!CAUTION] The deployed platform does not meet that policy today. As of 2026-08-19 the application runs in **Qatar** (`me-central1`) and the database in **the United States** (`us-east4`). The rule below is the target state; the gap is open, not closed.

What	[TARGET] where	[AS BUILT] where today	Why the target
Cloud SQL (Postgres)	<code>me-central2</code> (Dammam)	<code>us-east4</code> (N. Virginia) — <code>flygaca-sa-instance</code> , PG 18; plus <code>flygaca-fdc</code> in <code>me-west1</code> (Tel Aviv)	Holds every account, profile, logbook and entitlement row.
Cloud Run (the Fly GACA API + Captain Adel gateway)	<code>me-central2</code> (Dammam)	<code>me-central1</code> (Doha, Qatar) — 14 Functions-era services	Receives personal data (user queries).
Cloud Storage (SPA bucket + corpus bucket)	<code>me-central2</code> (Dammam)	Served from the <code>flygaca-sa</code> project	Public artifacts, but kept co-regional for latency and simplicity.

What	[TARGET] where	[AS BUILT] where today	Why the target
Public corpus pipeline / RAG chunk build / evals / staging	EU VPS (Paris) — Hostinger	Same — unchanged	Public data only ; explicitly never personal data.
ALLaM GPU endpoint (planned)	KSA / in-Kingdom GPU host	Not deployed	Same reason as the gateway — see <code>ay2m/Captain-Adel deploy/allam-vllm.md</code> .

[!WARNING] `me-centra11` is Doha, Qatar — it is NOT in-Kingdom. Never describe it as Riyadh, Saudi or "the Kingdom box". Any doc wording that pairs `me-centra11` with a PDPL in-Kingdom claim is wrong on its face — **including any doc describing the platform as it runs today, which is exactly where it runs.** `me-centra12` (Dammam) is the canonical target, and it is not reachable until Google grants the account access to the region.

Cloud Run and Cloud SQL must stay in the **same** region: the service reaches the database over a Cloud SQL unix socket (`/cloudsql/PROJECT:REGION:INSTANCE`), so there is no IP allowlist and no proxy sidecar. There is no region constant in the codebase to keep in sync — both are regional resources fixed at deploy time.

[TARGET] Google Cloud

Everything in this table is what the deploy runbook *will* create. Today's live project is `flygaca-sa` and today's live resources are in the `[AS BUILT]` table at the top of this page.

Item	Value
Project	Set at deploy time; the runbook provisions <code>flygaca</code> (the old <code>flygaca-firebase</code> / <code>flygaca-com</code> / <code>fly-gaca-495116</code> projects are dead — do not cite them). Production today is <code>flygaca-sa</code>.
Region	<code>me-centra12</code> (Dammam) for Cloud Run, Cloud SQL and the buckets — blocked: not granted to the account
Cloud Run service	<code>flygaca-api</code> — built from <code>server/</code> at the repo root. Not deployed anywhere yet.
Cloud SQL instance	<code>flygaca-db</code> (POSTGRES_16); database <code>flygaca</code> ; schema in <code>server/migrations/</code> . Live instance today is <code>flygaca-sa-instance</code> (PG 18, us-east4).
SPA bucket	<code>flygaca-web</code> behind an HTTPS load balancer (bucket backend on <code>/*</code> , serverless NEG on <code>/api/*</code>)
Corpus bucket	The regulatory JSON corpus is offloaded to a bucket and served network-first
Secrets	Secret Manager: <code>session-secret</code> , <code>database-url</code> , <code>google-oauth-secret</code> , <code>genai-api-key</code> , <code>moyasar-secret-key</code> , <code>moyasar-webhook</code> , <code>mail-api-key</code> , <code>cron-secret</code>
Scheduler	<code>flygaca-renewals</code> — daily <code>POST /api/billing/renew</code> with <code>X-Cron-Secret</code>

Item	Value
Config surface	Plain env vars on the Cloud Run revision (<code>server/src/config.ts</code> is the only place the server reads <code>process.env</code>)

Full provisioning sequence: `docs/RUNBOOK-deploy.md` in the product repo, mirrored here as `runbooks/runbook-deploy.md`. Secret and price **names** are mapped in `secrets-and-keys-placement.md`.

Mirrors (all proxy back to the same Cloud Run origin)

A Cloudflare Worker and the Netlify / Vercel mirrors each serve the same `dist/` build and rewrite `/api/*` back to `https://api.flygaca.com` as a same-origin proxy — so chat and account keep working and the strict CSP (`connect-src 'self'`) never changes. Keep any new API surface under `/api/*` for that to hold. The mirrors `X-Robots-Tag: noindex` any host that isn't `flygaca.com`.

EU VPS (Paris) — public-data box

Item	Value
Provider	Hostinger KVM 2 (2 vCPU / 8 GB / 100 GB)
IP	72.62.20.20
Hostname	srv1209075.hstgr.cloud
Region	Paris, Île-de-France (Hostinger, AS47583)
OS	Ubuntu 24.04 LTS, hardened (SSH key-only, ufw, fail2ban)
Role	Corpus ingest / RAG chunk build / eval harness / staging — public data only

GitHub

All product code is under the `ay2m` account. There is no `FLYGACA` org — those paths are legacy redirects.

Repo		What it is
<code>ay2m/FlyGACA</code>	private	The product. The bilingual web app (React 19 + Vite) <i>and</i> its Express backend on Cloud Run. The regulatory corpus and content pipelines live here too.
<code>ay2m/Captain-Adel</code>	private	The AI flight instructor service behind captadel.com
<code>ay2m/FlyGACA-ios</code>	public	The native SwiftUI family — one shared package, one App Store app per exam module
<code>ay2m/Office</code>	private	This repo — the company documents tree

Repo		What it is
ay2m/FlyGACA-app	public, archived	The retired predecessor of ay2m/FlyGACA . Read-only. Do not cite it as current.

[!NOTE] The six per-module App Store metadata repos ([PPL](#) , [CPL](#) , [IR](#) , [ATPL](#) , [ELPT](#) , [AIP](#)) that older docs reference **do not exist**. Per-app metadata lives inside [ay2m/FlyGACA-ios](#) .

Sources

- [FlyGACA/docs/RUNBOOK-deploy.md](#) — the authoritative provisioning + deploy sequence
- [FlyGACA/docs/DATA-HOSTING.md](#) — how the corpus bucket is served
- [FlyGACA/server/migrations/0001_init.sql](#) — the schema, with each table's Firestore-era ancestor noted
- [00-strategy/phase0.md](#) — P0-2 (domain decisions), P0-6 (VPS + PDPL boundary)
- [flygaca-resume-briefing-2026-05-23.md](#) — hosting + PDPL summary (pre-dates the Cloud Run port)
- [gcloud](#) inventory of project [flygaca-sa](#) , run **2026-08-19** — the source for the [\[AS BUILT\]](#) table